



INF2006 Cloud Computing & Big Data

Internship Matcher

Done by: Group P1-G5

Student Name	Student ID
Chye Zhi Hao	2500961
Goh Jia Xin	2500020
Chan Si Ying, Nasya	2502538
Yu Chuying	2502258
Liao Xue E	2501777

TABLE OF CONTENTS

1. Problem, users and success criteria.....	3
2. Solution overview and architecture.....	3
3. Cloud service/deployment choices and trade-offs.....	5
4. Implementation, data design and security controls.....	6
5. Analytics or AI/ML feature: data, method, evaluation and limitations.....	7
6. Testing, scalability/resilience and monitoring results.....	9
7. Cost, sustainability and operational considerations.....	11
8. Team contribution, ethical considerations and reflection.....	12
References.....	13
Appendix: evidence index.....	13

1. Problem, users and success criteria

Problem. Students searching for internships face long, inconsistent listings. Keyword search only finds exact words, so a student who writes "built REST services in Flask" is not matched to a listing asking for "backend API development in Python". Students therefore either read every listing or miss relevant ones. A useful tool must also handle résumés carefully, because they contain names, phone numbers, addresses and identity numbers.

Users. The primary users are university students looking for internships in Singapore. The secondary user is the team acting as operator, who imports the job catalogue and runs the service.

Workflow supported. A student signs in with Google, browses and searches the internship catalogue, and may upload a résumé PDF. The service extracts the text, removes personal data, and shows an editable draft. Only after the student approves the draft is it embedded and used for matching. The student then sees internships ranked by how well their approved experience covers each job's required skills, with the supporting résumé passage shown for every requirement, and can apply on the original site.

Success criteria (from the MVP requirements, docs/handoff/MVP_PRD.md):

Area	Criterion	Evidence
Workflow	Sign-in, upload, review, save, match, search and refresh work on the deployed service	evidence/test-functional.md F1
Validation	Oversized, missing, scanned, encrypted or over-length PDFs are rejected clearly without altering saved data	F2, pipeline tests
Privacy	Names, emails, phones, addresses, NRIC/FIN IDs and URLs are removed before review; only approved text is embedded	evidence/threat-control-map.md T7
Matching quality	Requirement-level ranking is measured against a keyword baseline on held-out data and the result is reported honestly	evidence/test-data-ai.md
Resilience	Loss of a web/API instance is detected and recovered without losing saved data	evidence/test-resilience.md
Security	Unauthenticated, cross-site and direct-network access is refused	evidence/test-security.md
Cost	Runs within a 50 USD Learner Lab allowance per member	Section 7

2. Solution overview and architecture

Figure 1 (evidence/architecture.png) shows the deployed architecture. Labels are the CloudFormation logical IDs in `src/infra/cloudformation/`, and the deployed state was captured read-only on 3 October 2026 (evidence/cloud-capture-2026-10-03/).

Internship Matcher — deployed AWS architecture (us-east-1)

Labels are CloudFormation logical IDs from `src/infra/cloudformation/private-base-ingress-http-api-private-apps.yaml`. Deployed state captured read-only 2026-10-03; evidence/cloud-capture-2026-10-03/.

Public Internet (untrusted)

- Student browser**
 - Read SPA
 - HttpOnly Secure session cookie + CSRF token
 - synthetic résumé PDFs only
- Google Identity Services (GaaS)**
 - GSI sign-in issues an ID token; API verifies issuer, audience, expiry

AWS account - Learner Lab - us-east-1

Vpc - private trust boundary (no instance has a public IP)

- PublicSubnetA - us-east-1a**
 - NatGatewayA** (outbound only → InternetGateway)
- PublicSubnetB - us-east-1b**
 - NatGatewayB** (outbound only → InternetGateway)
- AppSubnetA - us-east-1a (private)**
 - AppSubnetB - us-east-1b (private)**
 - VpcLink ENIs**
 - VpcLinkSecurityGroup: no inbound; egress TCP 443 to InternalAlbSecurityGroup only
 - InternalAlb - InternalAlbHttpsListener**
 - InternalAlb - HTTPS 443 - ELBSecurityPolicy-TLS13-1-2-2021-06 - inbound only from VpcLinkSecurityGroup
 - AppHttpsTargetGroup: HTTPS 8443, health check GET /health/ready every 30 s (2 failures = unhealthy)
 - App instance A - t3.xsmall**
 - web: Nginx - SPA, TLS 8443, 6 MB body cap, sanitised access log (no query string)
 - api: FastAPI - Google token check, DB sessions, CSRF, ownership, validation, matching, outbox publisher
 - AppTargetSecurityGroup: 8443 from ALB only
 - App instance B - t3.xsmall**
 - same containers and controls as A (AppLaunchTemplate, digest-planned ECR images, IMDSv2 required)
 - AppSecurityGroup: no inbound rules
 - AppTargetSecurityGroup: 8443 from ALB only
 - AppAutoScalingGroup: 2 instances - ELB health checks**
 - WorkerPerformanceRecovery - t3.medium**
 - extraction service: pdfjambler + Presidio
 - redaction (sandboxed child process)
 - embedding service: ai-MiniLM-L6-v2
 - WorkerSecurityGroup: no inbound; egress 5432 to DB, 443 and DNS only
 - DatabaseSubnetA/B** (no Internet route)
 - Database - RDS PostgreSQL 16.15 + pgvector**
 - db.t3.micro, Single-AZ (no standby), storage encrypted, not publicly accessible, 1-day automated backups
 - users, sessions, résumé profiles, jobs, requirement vectors, processing tasks + transactional outbox
 - DatabaseSecurityGroup: 5432 from AppSecurityGroup, WorkerSecurityGroup
 - S3Endpoint** (Gateway VPC endpoint on private route tables (S3 traffic stays on AWS network))

AWS regional managed services (IAM-scoped)

- Amazon S3 (SSE)**
 - ExtractionQueue - ExtractionDq
 - EmbeddingQueue - EmbeddingDq
 - redrive after 5 receives; 900 s visibility
 - API outbox publishes; worker long-polls (both via NAT, HTTPS 443)
- TemporaryInputBucket (S3)**
 - temporary résumé-PDF input only, never served
 - 1-day expiry - SSE-S3 - public access blocked
 - TemporaryInputBucketToOnlyBucket policy
 - objects deleted after terminal extraction
- AWS Secrets Manager**
 - in2006/db-urtime, db-migrator, db-master,
 - api-signing-key, duckdns-token
 - read at boot into root-only env files;
 - never in images, repo or logs
- Amazon ECR**
 - in2006/ecr-out, cloud-web, cloud-bootstrap
 - images pulled by immutable sha256 digest
- Amazon CloudWatch**
 - Logs (30-day retention): ApiLogGroup, WebLogGroup, WorkerLogGroup, API Gateway AccessLogGroup
 - Alarms: AppInServiceAlarm, ExtractionAppAlarm, EmbeddingAppAlarm, ExtractionDqAlarm
 - LoggingDqAlarm (no notification target)
 - Logs Insights: status/trace and error queries (`src/infra/scripts/capture-logs-insights.py`)
- AWS Systems Manager**
 - Session Manager admin access: no SSH keys, no bastion, no inbound management port
- IAM: LabRole via LabInstanceProfile**
 - Learner Lab forbids custom roles, so all instances share one lab role; least privilege is enforced by network rules and restricted database roles instead
- Certificate (ACM import)**
 - Let's Encrypt cert for the ALB listener; DNS-01 validation via DuckDNS; renewal timer on worker (hostname used for TLS identity, not browsing)

Data flows

- Browser → HttpApi over HTTPS: SPA assets and /api on one origin; session cookie plus CSRF header on state-changing requests.
- HttpApi → VpcLink → InternalAlb over HTTPS 443 for private integration; API Gateway verifies the listener certificate name).
- InternalAlb → app instances over HTTPS 8443 (encrypted; the ALB does not validate the target certificate).
- API → Database (TCP 5432): sessions, profiles, catalogue, matches; task + outbox rows commit atomically.
- API → TemporaryInputBucket via S3Endpoint: uploaded PDF stored temporarily.
- Private route tables send 0.0.0.0/0 to NatGatewayA/B: outbox → S3, worker polling, Secrets Manager, ECR, SSM, CloudWatch, Google certificates.
- Worker reads the PDF via S3Endpoint, writes the redacted draft and requirement vectors to the Database, then deletes the temporary object.
- Container logs and metrics → CloudWatch.
- Google sign-in (ID token verified server-side).

Trust boundaries: red = untrusted Internet; orange = AWS account; purple = VPC with private app subnets; blue = database subnets with no Internet route. Only HttpApi is reachable from the Internet.

task and an outbox row, then returns HTTP 202. (2) The outbox publisher sends the task to the extraction queue. (3) The worker reads the PDF, extracts and redacts text, writes the draft to the database and deletes the PDF. (4) The browser polls the task and shows the draft. (5) On save, the API stores the approved content and queues an embedding task; the worker writes requirement-matching vectors. (6) GET /api/matches ranks jobs in PostgreSQL using those vectors.

Trust boundaries. The Internet is untrusted. Only `HttpApi` accepts inbound traffic; the ALB is internal and accepts only the VPC Link security group; app instances accept port 8443 only from the ALB; the worker and the database accept nothing from the Internet, and the database accepts port 5432 only from the app and worker security groups. The database subnets have no route to the Internet.

3. Cloud service/deployment choices and trade-offs

Service model. We chose IaaS compute with managed PaaS services around it. The team manages the EC2 guest operating system, Docker runtime, Nginx and FastAPI; AWS manages the database engine (RDS, a DBaaS), queues (SQS), object storage (S3), the API front door (API Gateway), load balancing (ALB) and secrets. Google Identity Services is an external SaaS identity provider. The IaaS/PaaS boundary therefore sits at the VM: everything inside the instances is ours to patch and configure, and everything outside is a managed service with an SLA.

Deployment model. Public cloud, single AWS region (us-east-1, required by the Learner Lab), two Availability Zones for the web/API tier, deployed as three CloudFormation stacks (`inf2006-private-base-retry1`, `inf2006-private-ingress`, `inf2006-private-app`). Local Docker Compose reproduces the same application without AWS, so markers can run it offline.

Why EC2 rather than a more managed compute service. The extraction and embedding steps load spaCy, Presidio and a PyTorch model of several hundred megabytes and run a sandboxed child process with a 60-second deadline. That fits a long-running VM with predictable memory. It does not fit short-lived functions, and the Learner Lab forbids the custom IAM roles that many managed container setups expect.

Alternatives considered:

Alternative	Benefit	Why not selected
A. One VM running web, API and PostgreSQL	Cheapest; simplest network	Database on the public host; no isolation, backup or recovery separation; one failure takes everything down. Our first foundation (27 Sept) already moved the database to RDS.
B. One public EC2 host plus private RDS (our 27 Sept foundation)	Simple; deployed and tested	Single web/API host with a public IP; PDF processing ran on the API host with one processing slot, so a second upload got a busy response and a host failure meant an outage. Replaced by the current design.
C. AWS Lambda functions + DynamoDB	Pay per request; no servers	Model loading and PDF processing exceed comfortable

		function limits; vector search would need another service; rewriting the data layer for DynamoDB removes pgvector ranking.
D. Container service (ECS/Fargate) with RDS	Less host management	Feasible, but the Learner Lab IAM constraints and time budget favoured the EC2 model the team already ran; would be the next step.
E. Cognito instead of Google sign-in	AWS-native identity	Students already have Google accounts; the server verifies Google ID tokens directly. Deferred.
F. PostgreSQL table as the job queue instead of SQS	No extra service	SQS gives managed dead-letter queues and backlog/age metrics for alarms; the PostgreSQL-backed queue is kept for local runs.

Main trade-offs accepted. Two NAT gateways cost about a third of the hourly bill but keep each AZ's outbound path independent; a single NAT gateway or interface endpoints would be cheaper. RDS is Single-AZ: a Multi-AZ standby would roughly double database cost, which the budget does not justify for a demonstration. API Gateway in front of an internal ALB adds a hop and a 30-second integration timeout, but removes every public IP and gives us an HTTPS origin without managing a public certificate on the browser side.

Assumptions and expected workload. Demonstration scale: tens of students, a catalogue of a few hundred listings (247 deployed), and occasional bursts of uploads (planning figure: 50 uploads queued). Browsing and matching are cheap database queries; uploads are expensive and are therefore made asynchronous so a burst becomes a queue backlog rather than failed requests. Test data is synthetic or public job listings, with one disclosed exception: one team member also uploaded their own résumé to the deployed service and reviewed the redacted draft, without saving it, and then deleted it. No résumé content is in the submission package.

4. Implementation, data design and security controls

Application. The backend is FastAPI with SQLAlchemy and Alembic migrations (`src/backend/`); the frontend is React 19 and TypeScript built with Vite (`src/frontend/`). Uploads return 202 and the browser polls a task resource. Saves carry the current revision and an idempotency key, so retries after a network failure apply once and a stale tab cannot overwrite newer work (HTTP 409). The task and its outbox message are written in the same transaction as the business change, so a crash cannot lose a queued task or send one for data that was never saved; workers hold fenced leases so a slow worker cannot overwrite a newer result.

Data design. Eleven PostgreSQL tables are defined by migrations: `users`, `sessions`, `resume_profiles`, `resume_chunks`, `save_operations`, `jobs`, `job_requirements`, `requirement_embeddings`, `app_state`, `processing_tasks` and `processing_outbox`. Each job requirement is stored separately with its source quote and any "OR" alternatives, and each requirement and alternative has a 384-dimension vector. Résumé chunks are stored with

their vectors and a model version. The catalogue import (`app.catalogue.import_jobs`) validates the entire batch, is idempotent (a repeat import of the 247 jobs reported zero changes and zero recomputed embeddings) and reuses vectors for unchanged text. Field definitions are in `data/DATA_DICTIONARY.md`.

Input validation. Limits are enforced in layers: Nginx rejects bodies over 6 MiB; the API limits multipart bodies to 6 MiB and JSON to 256 KiB; PDFs must be at most 5 MiB and 10 pages and must contain extractable text, so encrypted, scanned and non-PDF files are rejected with specific error codes.

Security controls (full mapping in `evidence/threat-control-map.md`, 13 threats):

- *Authentication:* Google ID tokens are verified server-side (signature, issuer, audience, expiry) with a one-time nonce; the server then issues its own database-backed session in an `HttpOnly`, `Secure`, `SameSite=Lax` cookie. Only a SHA-256 hash of the session token is stored, and sessions expire after 30 minutes idle or 8 hours absolute.
- *Authorisation:* the user is always taken from the session; no route accepts a user ID, so one student cannot name another student's data.
- *CSRF:* every state-changing request must come from the allowed Origin and carry the session-bound CSRF token.
- *Network restriction:* as described under trust boundaries; no instance has a public IP; IMDSv2 is required.
- *Secrets:* stored in Secrets Manager, read at boot into root-only (0600) environment files, never in images, templates, logs or the repository.
- *Data protection:* RDS, S3 and SQS are encrypted at rest; every network hop uses TLS; S3 blocks public access, enforces TLS and expires objects after one day.
- *Least privilege:* the Learner Lab does not allow custom IAM roles, so all instances use the lab's `LabRole`. We compensated with network isolation and with two restricted PostgreSQL roles (`migrator` and `runtime`) that cannot create databases or roles or bypass row security. This is a genuine limitation, not a solved problem.

5. Analytics or AI/ML feature: data, method, evaluation and limitations

Purpose. The data-driven feature is the recommender: it decides which internships a student sees first and explains why, which is the core decision in the workflow.

Data. Two datasets are used.

(1) A synthetic evaluation set (`data/evaluation/`): 30 internship listings with hand-written requirements, 10 synthetic résumé profiles split into 5 development and 5 held-out, and 300 relevance labels on a 0/1/2 scale defined in `LABELLING_CRITERIA.md`. 90 labels (all 30 jobs for held-out profiles P06 to P08) were supplied by a team member (process recorded as blind; independence not verified); the other 210 were drafted by an AI assistant from the written criteria before any model output existed. The headline table below uses the AI-drafted labels for all five held-out profiles, and a separate check re-scores P06 to P08 with the human labels.

(2) The deployed catalogue of 247 real Singapore internship listings collected on 1 October 2026 (78 via the JSearch API, 169 from LinkedIn via an Apify scraper), reviewed record by record with reasons for every exclusion. Because redistribution rights are not established, this catalogue is not in the submission; all reproducible results use the synthetic set.

Preprocessing. Résumés: text and layout extracted with pdfplumber, sections detected from headings and fonts, PII replaced with placeholders, then split into chunks within the model's token limit (the largest chunk was 108 of 240 allowed tokens; nothing was truncated). Jobs: each required skill becomes its own requirement row, quoted from the original description.

Method. Each requirement (and each "OR" alternative) and each résumé chunk is embedded with `all-MiniLM-L6-v2` at a pinned revision. A requirement's score is its best cosine similarity to any résumé chunk; a job's score is the mean over its required requirements, so a job scores highly only if most of its requirements are covered. Preferred skills and eligibility notes are shown but do not change ranking. The output is interpretable: for every requirement the student sees the closest résumé passage, and scores are presented as similarities, not as chances of being hired.

Evaluation. Precision@5 (strict counts only label 2; lenient counts labels 1–2) and NDCG@5 on the held-out profiles, compared with three baselines.

Method (held-out, MiniLM)	P@5 strict	P@5 lenient	NDCG@5
Requirement-level (deployed method)	0.640	0.760	0.922
Keyword / BM25 baseline	0.640	0.720	0.910
Pooled résumé embedding	0.560	0.640	0.771
Whole-résumé single vector (truncated)	0.560	0.680	0.808

On the 90 labels supplied by a team member (profiles P06–P08 only), the keyword baseline scored higher than our method: strict P@5 0.600 vs 0.533, lenient P@5 0.867 vs 0.800 and NDCG@5 0.920 vs 0.870. With three profiles this is a point estimate, but together with the headline table it means we cannot claim a measurable accuracy advantage over keyword search; what the method adds is requirement-level explanation. The best achievable strict P@5 on this set is 0.680. The requirement-level method matches BM25 on strict precision and is slightly better on lenient precision and NDCG; both clearly beat embedding the whole résumé as one vector. Mean scores rise with label (held-out: 0.212 for irrelevant, 0.291 partial, 0.420 relevant), so the scores carry signal. A second model (`bge-small-en-v1.5`) gave similar results and was not adopted. The evaluation was reproduced on 3 October on a different operating system and CPU with identical numbers.

Limitations and responsible use. The evaluation is small (5 held-out profiles, no confidence intervals), synthetic text is cleaner than real résumés, most labels are AI-drafted, and quality on the real catalogue was not measured because it has no labels. The advantage over BM25 is therefore not statistically established; the main benefit we can claim is explainable, requirement-by-requirement matching with comparable precision. PII detection makes mistakes in both directions; in testing "Cloud Computing" was wrongly removed as a person's name, which is why the student reviews the draft before anything is stored or embedded. The model is English-only and could disadvantage students who describe skills in unusual wording; ranking explains rather than decides, and the student always sees every listing.

6. Testing, scalability/resilience and monitoring results

Four required tests were run, each recorded with objective, steps, expected and actual results, date and artefacts.

Test	Result	Record
Functional workflow	PASS: full journey on AWS — sign-in, upload, review, first save 12.58 s on a cold privacy check (single observation, not a benchmark), 247 ranked matches, search, refresh, offline retry with exactly one revision increment. Validation: 413 for oversized requests and PDFs, 415 for missing file, saved data unchanged.	evidence/test-functional.md
Security	PASS: without a session, /api/me, /api/resume, /api/matches and uploads returned 401; cross-site and missing-Origin state changes returned 403 CSRF_INVALID; captured configuration shows no public IPs, an internal ALB and database access only from app/worker security groups. 133 backend tests (including CSRF, Google-token and ownership tests) passed. A live check with two synthetic users on the local stack passed 29/29, and the same ownership, session and CSRF checks passed 26/26 on the deployed site with two real accounts, including account A's attempts to read and delete account B's real extraction task (evidence/test-security-live-local-2026-10-03.md, evidence/test-security-live-cloud-real-task.md). On the public endpoint, plain HTTP is not served, TLS 1.2 and 1.3 work and TLS 1.0 and 1.1 are refused (evidence/test-security-tls-2026-10-03.md).	evidence/test-security.md
Data/AI	PASS: Completed and reproduced exactly on a second machine; no clear advantage over the keyword baseline (see section 5).	evidence/test-data-ai.md
Resilience	PASS: two real failures and one deliberate instance termination recovered automatically (below); a controlled target withdrawal kept the service available; a point-in-time database restore matched the source.	evidence/test-resilience.md

Resilience mechanisms and observed results. The web/API tier runs two instances behind the load balancer, with ELB health checks on /health/ready (which queries the database) and Auto Scaling keeping two in service. Uploads are queued in SQS, so a burst or a worker outage delays work instead of losing it; messages failing five times move to a dead-letter queue. On 3 October the Auto Scaling activity history recorded two unplanned events in the morning, and a third in the afternoon that is described after the controlled test:

- 02:42 UTC: one instance failed the health check shortly after deployment; Auto Scaling terminated it and launched a replacement within seconds, with no operator action.
- 05:23–05:31 UTC: a Learner Lab session restart stopped both instances. Auto Scaling detected this through EC2 health checks; launches failed while the lab was unavailable, then two replacements started at 05:28:43. The in-service alarm entered ALARM at 05:25:59 and cleared at 05:30:59, about seven minutes from the first stop. The service had a full outage during this window — two instances protect against one instance or one zone failing, not against both being stopped.

Controlled test (3 October, 16:04 UTC). With both targets healthy we terminated one instance and polled the public endpoint every 3 s. Two of 290 probes failed (a 502 and a timeout in the first ~10 s, while the load balancer still routed to the dying target); every later probe returned 200. Auto Scaling detected the loss in about 45 s and launched a replacement, which became healthy after about 9 minutes with no operator action; the service ran on one instance in between.

A defect this testing found. After a second Learner Lab restart (about 15:26 UTC) the public endpoint returned HTTP 502 on every path (observed at 15:33 UTC) until the first instance became healthy at about 15:40 UTC, an outage of roughly ten minutes or more that we did not time precisely. Replacements in one zone were then repeatedly terminated before becoming healthy: bootstrap takes about 348 s and the load balancer needs several passing checks, but the health-check grace period was 420 s. The service stayed up only on the other zone's instance. We raised the grace period to 900 s on the running group and in the template (the deployed CloudFormation stack has not yet been updated to match), after which replacements reached healthy and no further terminations followed.

Database restore. We restored the RDS automated backup to a temporary private, encrypted instance (about 9 minutes) and compared it with the source from inside the application's network: same 12 tables, the pgvector extension and identical row counts. This shows a recent-state restore works; row contents, an application cut-over and recovery from older corruption were not tested. The runtime database role cannot read the migration table, as intended.

Monitoring. All container logs go to CloudWatch Logs with 30-day retention; Nginx logs omit query strings, cookies and headers. A Logs Insights query over the 24 hours to 07:13 UTC on 3 October (before the afternoon outage described above) (evidence/monitoring.md) found 2,269 requests: 90% were load-balancer health checks and the rest matched the acceptance session. Every 4xx response was explained by a deliberate test or an expected state, and there were no 5xx responses. Five alarms watch in-service instance count, queue message age and dead-letter queues.

Failed tests and improvements. Four local pipeline tests had failed because they created the processing service without the worker type added during the asynchronous redesign; the production code was correct. We fixed the tests and the pipeline suite now passes (220). A stale local test image initially broke the backend suite until rebuilt. Not tested: cloud load testing, alarm notifications (no SNS action is configured), and the queue alarms firing.

Planned improvements: add SNS notifications, put the worker in its own Auto Scaling group scaled on queue age, move the database to Multi-AZ, and add security headers.

7. Cost, sustainability and operational considerations

Cost. Cost Explorer reports zero in the Learner Lab, so we estimated cost from the captured inventory and on-demand list prices

(evidence/cloud-capture-2026-10-03/10-resource-inventory-and-cost.txt):

Item	USD/hour
2 × t3.small web/API	0.042
1 × t3.medium worker	0.042
2 × NAT gateway	0.090
RDS db.t3.micro Single-AZ	0.018
Internal ALB	0.023
Public IPv4 addresses (5)	0.025
Storage, secrets, registry, queues, logs	~0.017
Total while running	~0.26 (≈ 6.2/day)

At this rate a 50 USD allowance lasts about eight days of continuous running. Cost controls used: smallest instance classes that fit the models, Single-AZ RDS with 20 GiB storage, a one-day S3 lifecycle, 30-day log retention, an S3 gateway endpoint (free) instead of NAT for S3 traffic, and CloudFormation so the stack can be deleted and recreated. The largest saving available is replacing two NAT gateways with one (about 0.045/hour) at the cost of zone independence. Ending a lab session stops EC2 but not NAT gateways, the load balancer or Elastic IPs, so stacks must be deleted, not just paused. A post-deployment inventory found leftovers from failed attempts — three unused Elastic IPs, one RDS snapshot and one rolled-back stack — that should be removed.

Sustainability. Embeddings are computed once per job requirement and reused across imports and students (re-importing 247 jobs recomputed zero embeddings); the model is small and CPU-only, so no GPU is needed; the worker processes one task at a time instead of keeping idle capacity; and temporary files expire automatically.

Operations. Deployment is scripted (docs/PRIVATE_CLOUD_ROLLOUT.md) with images pinned by digest. Administration uses Session Manager without SSH keys. The TLS certificate for the internal listener comes from Let's Encrypt via DNS validation and renews on a timer; a complete renewal cycle has not yet been observed. One reproducibility gap remains: a renewal-hook fix was applied directly to the running worker and is committed to the repository, but the published images predate it.

8. Team contribution, ethical considerations and reflection

Contributions. Jiaxin built authentication, sessions, the data model and migrations; Chuying built PDF extraction, privacy redaction, embeddings, matching and the evaluation, and supplied 90 relevance labels; Xue E built the catalogue, search and job-details frontend; Nasya built the résumé upload, review and recommendation screens and their tests; Zhihao led system design, integration, cloud deployment and evidence. Commit-level detail is in `TEAM_CONTRIBUTIONS.md`.

Use of AI tools. OpenAI Codex and Anthropic Claude assisted with code, infrastructure templates, the evaluation harness, synthetic fixtures and documentation, as declared in `AI_USE_DECLARATION.md`. AI output was accepted only after tests, source review or captured command output confirmed it; AI-drafted evaluation labels are reported as such rather than as human ground truth.

Ethical considerations. Résumés are sensitive, so the design minimises what is kept: the PDF is deleted after extraction, personal identifiers are removed before the student sees the draft, and only content the student approves is stored and embedded. Recommendations show their evidence and make no claim about hiring likelihood. Job listings were collected from public sources for a coursework demonstration and are not redistributed; scraping terms of service are a real concern, and a production service would use a licensed feed.

Reflection. As a team, the most valuable change was moving from one public server with synchronous, one-at-a-time processing to a private, queue-based design, so that uploads queue instead of being refused and no server is directly exposed. The hardest part was not building features but producing evidence a marker can verify without our AWS account. With more time we would replace the NAT gateways with VPC endpoints, repeat the restore drill with an application cut-over and gather real labelled data to test whether semantic matching beats keyword search on real résumés.

Zhi Hao: Building this project helped me develop a stronger understanding of cloud computing fundamentals and gain hands-on experience with AWS services. Beyond preparing me for the AWS Certified Cloud Practitioner certification, this experience has given me practical knowledge that I can apply to my startup journey.

SiYing: This project has changed my understanding of cloud computing and have learnt to appreciate the fundamentals, as well as gain exposure to AWS services. I can use the takeaways to better apply to my learning and real world applications.

Jia Xin: Working on this project has effectively bridged the concepts I learned in lectures and practiced in labs with solving an actual, real-world problem, turning abstract fundamentals into tangible, purpose-driven skills. I've come to see cloud services not just as technical tools to memorise, but as practical solutions that address real needs and create real value. This experience has given me greater confidence to apply what I've learned in future learning .

Xue E: I developed and refined the job catalogue, search, and job-details frontend, focusing on making internship information clearer and easier for students to explore. Through this project, I gained practical experience translating real user needs into a functional React/TypeScript interface and learned how frontend design must work closely with backend services and cloud deployment in a real-world application.

Chuying: I was responsible for the human review of our AI evaluation labels, blindly labelling 90 profile-job pairs (P06 to P08) with AI helping me set up the labelling spreadsheet. The evaluation showed our requirement-level method did not clearly beat the keyword baseline. This taught me that cloud design choices, such as running a small CPU-only model on a t3.medium worker, need measured evidence and honest limits instead of assumptions. With more time I would label all profiles with a second reviewer.

References

- AWS Documentation: Amazon API Gateway HTTP API private integrations; Elastic Load Balancing; Amazon EC2 Auto Scaling health checks; Amazon RDS for PostgreSQL extensions (pgvector); Amazon SQS dead-letter queues.
- Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.
- sentence-transformers/all-MiniLM-L6-v2 model card (Apache-2.0). Microsoft Presidio (MIT). pdfplumber (MIT). pgvector (PostgreSQL licence).
- Järvelin, K. & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. ACM TOIS 20(4).

Appendix: evidence index

All paths are relative to the submission root and listed in `project_manifest.yaml`; `evidence/README.md` indexes every artefact.